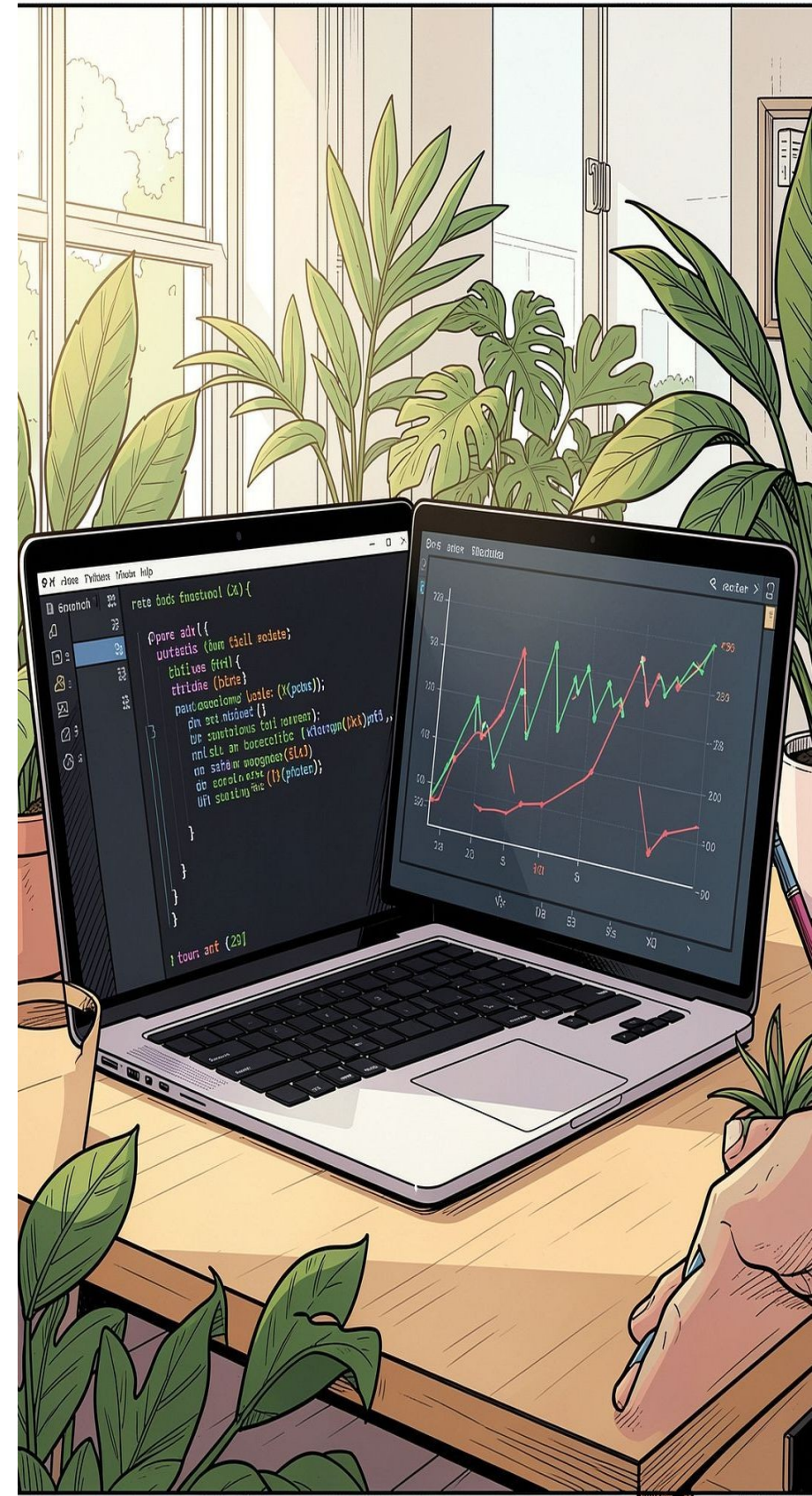
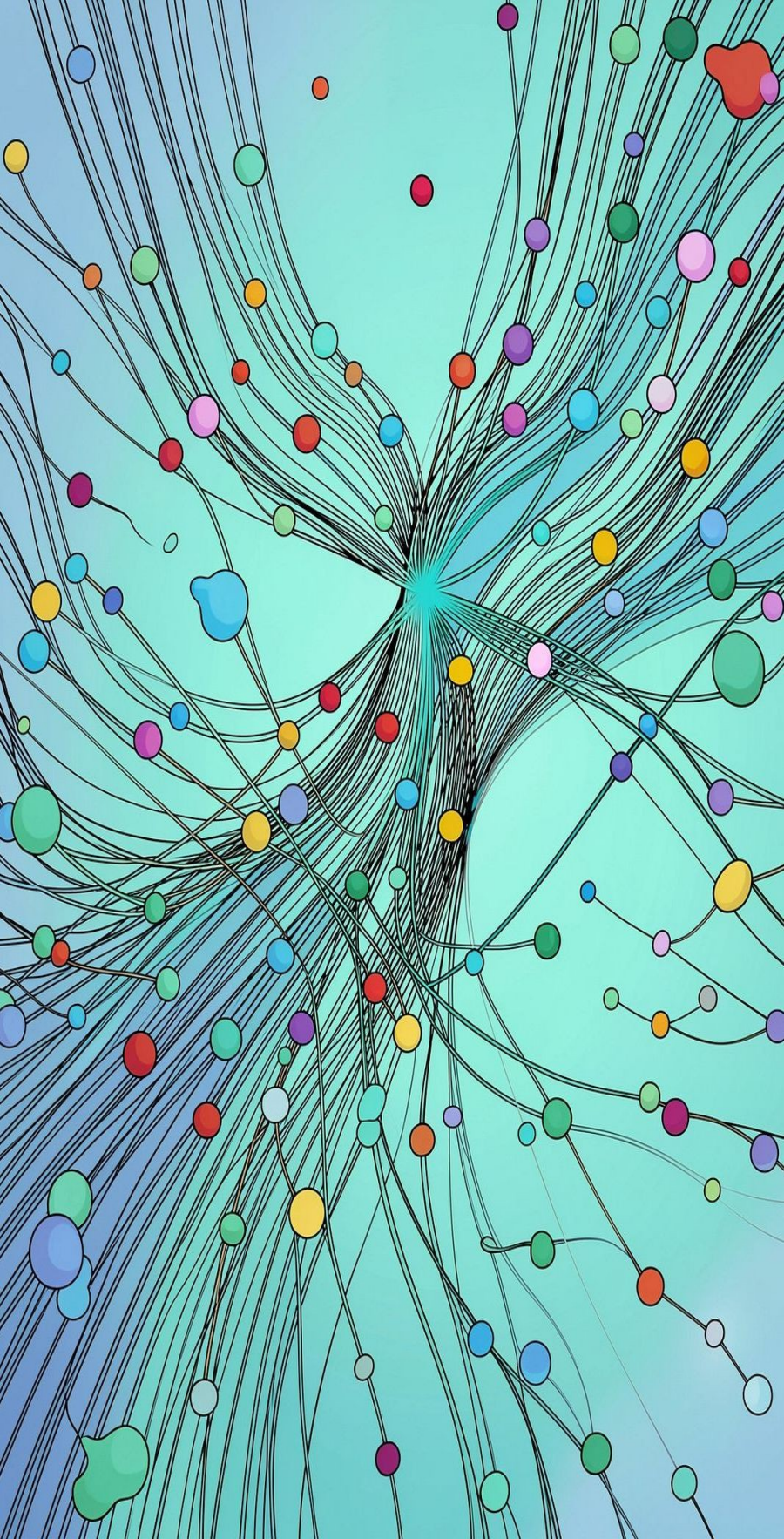


Data Preprocessing and Visualization Using Pandas and Matplotlib

Practical guide: preparing real-world data with Pandas and creating clear, persuasive visuals with Matplotlib. Compact, actionable techniques and two short case studies to illustrate workflow and best practices.

N.Harsha Vardhan
192324189
DSA0405-Fundamentals Of
Data Science





Why Data Preprocessing and Visualisation Matters

Preprocessing turns messy raw data into reliable inputs — improving model accuracy, reducing bias, and saving time. Visualisation clarifies patterns, aids decisions, and communicates results to stakeholders.



Trustworthy Data

Cleaning and validation reduce errors and avoid misleading analyses.



Insight Discovery

Plots reveal trends, outliers, and relationships not visible in tables.

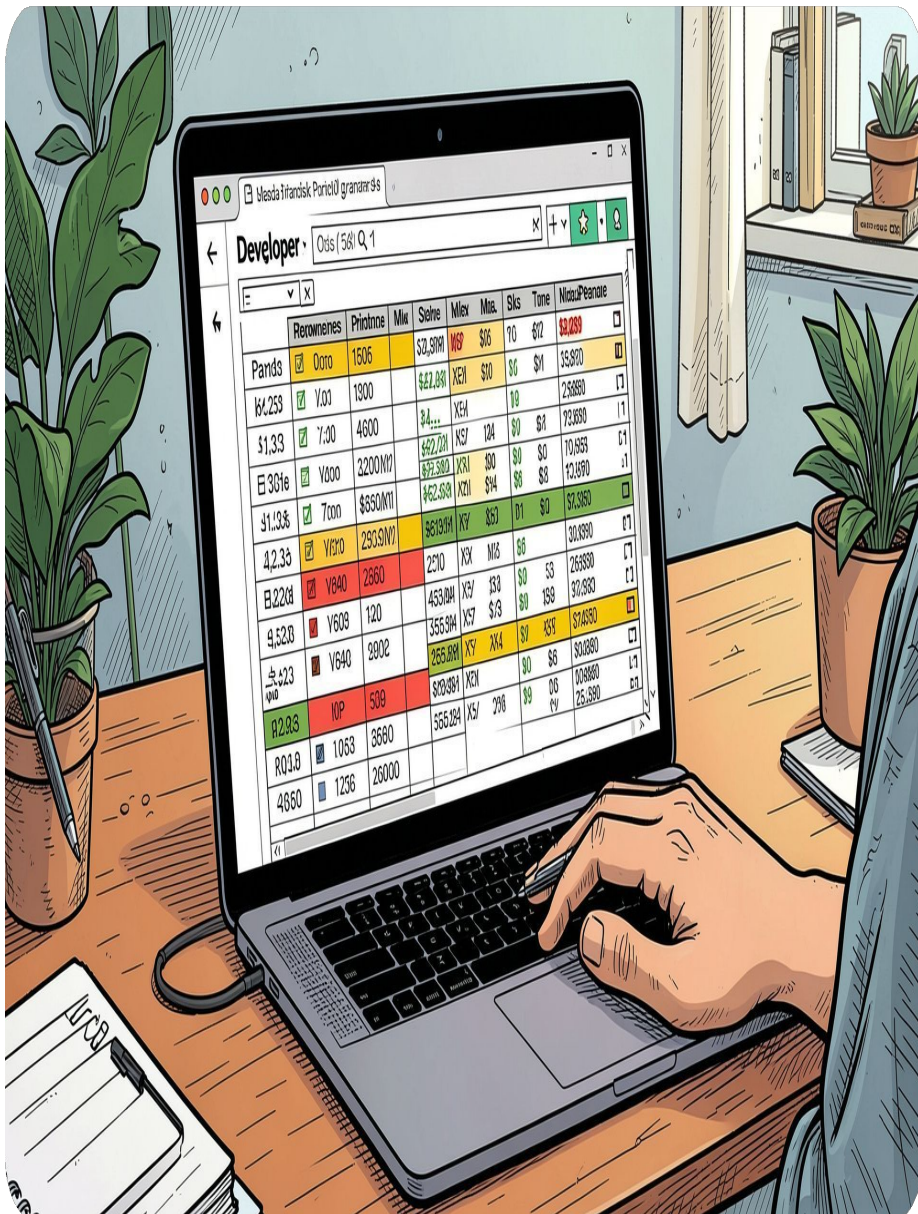


Decision Influence

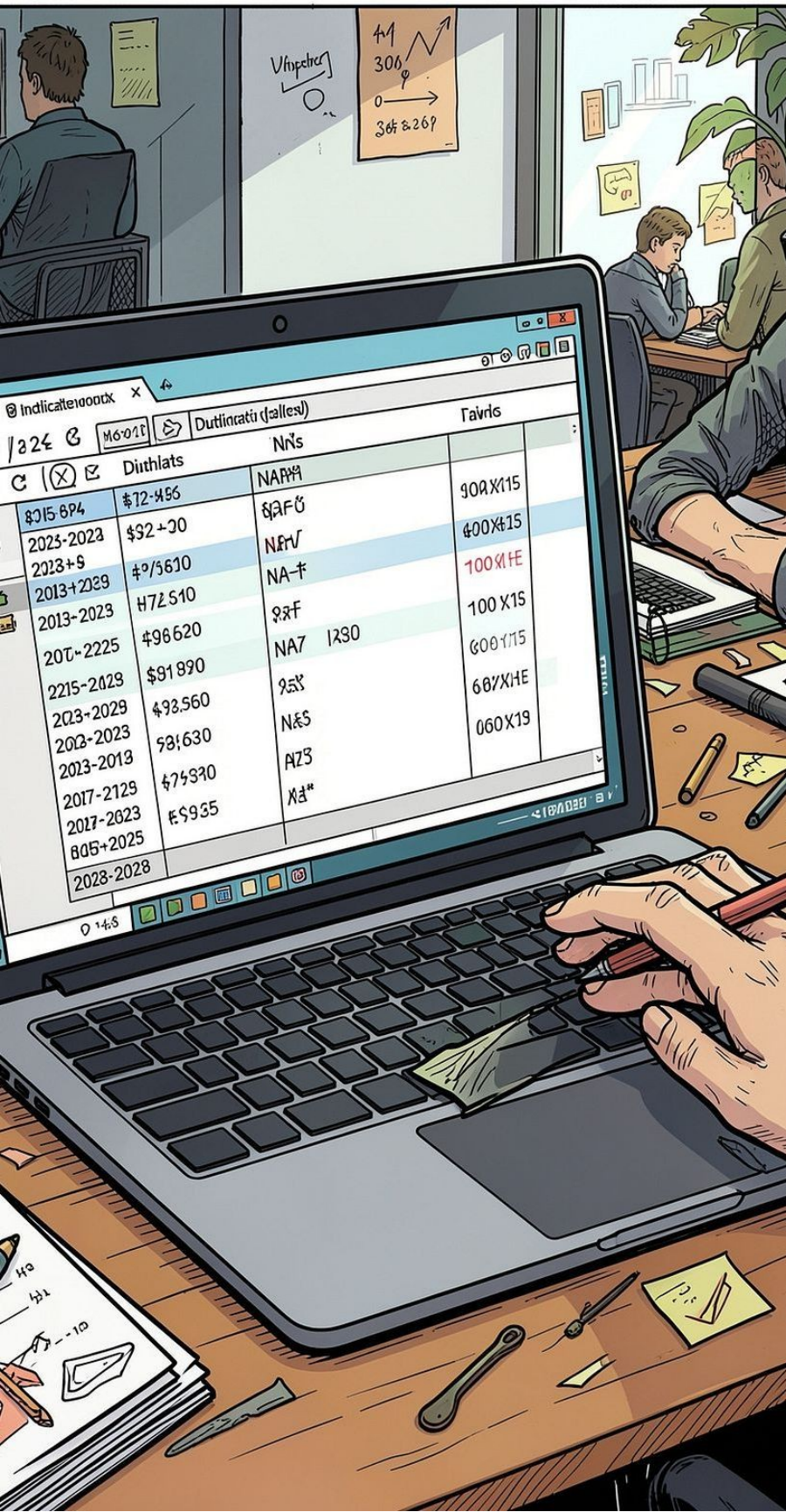
Effective visuals transform analysis into action for non-technical audiences.



Essential Pandas Operations: Cleaning & Structuring Data



- Inspect: `df.head()`, `df.info()`, `df.describe()` — quick health check
- Handle missing values: `df.dropna()`, `df.fillna(value)`, interpolation
- Type safety: `df.astype({'col':'int64'})` and `to_datetime` for time series
- Duplicates & outliers: `df.drop_duplicates()`, IQR or z-score for outlier detection
- Normalization & scaling: $(x - \text{mean})/\text{std}$ or MinMax for model prep
- Feature engineering: extract date parts, bin continuous variables, encode categories



Case Study 1: Real-world Data Cleaning with Pandas

A retail transactions dataset with mixed date formats, missing prices, and duplicate orders. Goal: produce a clean, analysis-ready table for weekly sales reporting.

Step 1 — Load & Inspect

Use `read_csv(parse_dates=['order_date'])` and `df.info()` to find issues.

Step 2 — Normalize Dates & Types

`df['order_date']=pd.to_datetime(df['order_date'], errors='coerce');`
fill or drop coerced rows.

Step 3 — Fill & Validate

Impute missing prices with median by category or flag for manual review.

Step 4 — Aggregate

Group by week: `df.resample('W', on='order_date').agg( {'sales':'sum','orders':'count'})`

Introduction to Matplotlib: Bringing Your Data to Life

Matplotlib is the foundational plotting library in Python. It offers fine-grained control over every plot element and integrates with Pandas for quick visual checks and publication-quality figures.

- Plot types: line, bar, scatter, histogram, boxplot, heatmap
- Styling: figure size, fonts, colors, subplots, annotations
- Integration: `DataFrame.plot(...)` for rapid exploration, `pyplot` for custom figures

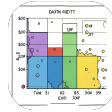


Common Matplotlib Plots: Unveiling Insights



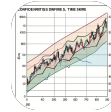
Histogram

Distribution, skewness, and bin selection for continuous variables.



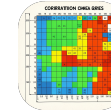
Boxplot

Compare medians and detect outliers across groups.



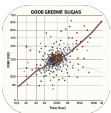
Time Series

Seasonality, trends, and rolling averages for temporal data.



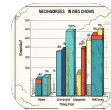
Heatmap

Visualise correlation matrices or pivot tables with color intensity.



Scatter

Relationship between two variables; add regression line for trend.



Bar / Stacked Bar

Compare categories or show composition across groups.

Case Study 2: Visualising Cleaned Data for Impact

Using the cleaned retail dataset, build a 3-panel figure: weekly sales (line), category share (pie), and price vs units (scatter). Focus on clarity and storytelling.

Panel 1 — Weekly Sales

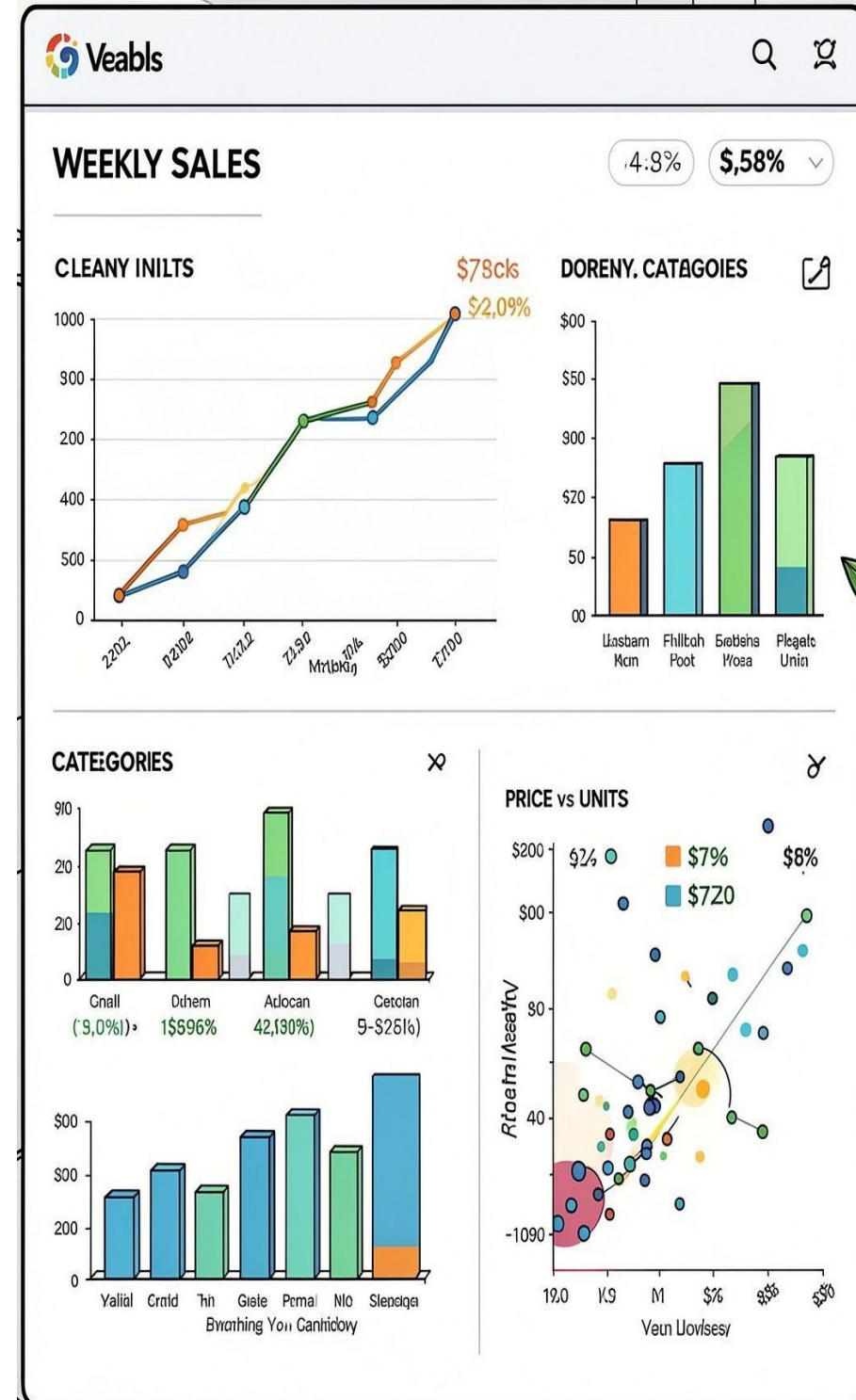
Line plot with rolling 4-week average and shaded confidence band to show trend and variability.

Panel 2 — Category Share

Pie or stacked bar showing relative revenue; combine small categories into "Other" for readability.

Panel 3 — Price vs Units

Scatter with point size = revenue, color by category, and regression line to highlight relationship.



Best Practices for Effective Data Presentation

- Start with a clear question — design visuals to answer it directly. Simplify: remove chartjunk, label axes, and avoid 3D effects.
- Choose appropriate scales (log vs linear) and show units.
- Use consistent color palette — use theme color `c6c9dcff` for highlights.
- Annotate key points and provide short captions for interpretation.
- Reproducibility: keep preprocessing code and plotting scripts together (notebooks or scripts).



Key Takeaways & Resources

Takeaways

Clean data + clear visuals = trustworthy insights. Automate preprocessing, document choices, and design visuals for the audience.

Resources

Official docs: pandas.pydata.org, matplotlib.org. Useful reads: Pandas Cookbook, Matplotlib for Data Visualization.

Next Steps

Apply these steps to a small dataset: audit, clean, visualise, and present — iterate based on stakeholder feedback.

